

```
print('Employee salary:', '{:,.2f}'.format(oEmployee1.payPerYear()))
print('Employee name:', oEmployee2.getName())
print('Employee salary:', '{:,.2f}'.format(oEmployee2.payPerYear()))
print()

# Call methods of the Manager object
managerName = oManager.getName()
print('Manager name:', managerName)

# Give the manager a bonus
print('Manager salary:', '{:,.2f}'.format(oManager.payPerYear(True)))
print(managerName, '(' + oManager.getTitle() + ')', 'direct reports:')
reportsList = oManager.getReports()
for oEmployee in reportsList:
    print('    ', oEmployee.getName(),
          '(' + oEmployee.getTitle() + ')')
```

When we run this code, we see the following output, as we would expect:

```
Employee name: Joe Schmoe
Employee salary: 33,280.00
Employee name: Chris Smith
Employee salary: 29,120.00

Manager name: Sue Jones
Sue Jones gets a bonus for good work
Manager salary: 60,500.00
Sue Jones (Pizza Restaurant Manager) direct reports:
    Joe Schmoe (Pizza Maker)
    Chris Smith (Cashier)
```

The Client's View of a Subclass

The discussion so far has been focused on the details of implementation. But classes can look different depending on whether you are the developer of a class or are writing code to use a class. Let's change focus and take a look at inheritance from the client's point of view. As far as client code is concerned, a subclass has all the functionality of the base class, plus anything defined in the subclass itself. It may help to think about the resulting collection of methods as layers of paint on a wall. When a client